Training Neural Networks

Problem Setup

Let $f_\Theta : \mathbb{R}^d \to \mathbb{R}^K$ be a neural network with parameters Θ. Given a training set $D_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ drawn i.i.d. from an unknown distribution ν over $\mathcal{X} \times \mathcal{Y}$, and a loss function $\ell : \mathcal{Y} \times \mathbb{R}^K \to \mathbb{R}_{\geq 0}$, we seek:

$$\Theta^* = \arg\min_\Theta \hat{\mathcal{R}}_n(\Theta) := \frac{1}{n} \sum_{i=1}^n \ell(y_i,\ f_\Theta(\mathbf{x}_i))$$

In linear regression, $\hat{\mathcal{R}}_n$ was convex and had a closed-form minimizer $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. For neural networks, neither property holds. The loss landscape $\hat{\mathcal{R}}_n(\Theta)$ is non-convex: it contains saddle points, local minima, and flat regions. There is no closed-form solution. We rely on iterative, gradient-based optimization.

Why non-convexity? A neural network composes nonlinear functions. Even a single hidden layer $f_\Theta(\mathbf{x}) = \mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{x})$ is non-convex in $(\mathbf{W}_1, \mathbf{W}_2)$ jointly, because the parameters interact multiplicatively. Permuting the hidden neurons gives distinct parameter vectors with identical function values, so any minimum is replicated by at least $m!$ equivalent minima (where m is the hidden width).

Gradient Descent and Variants

Full-Batch Gradient Descent

Starting from an initial Θ_0, gradient descent iterates:

$$\Theta_{t+1} = \Theta_t - \eta\, \nabla_\Theta \hat{\mathcal{R}}_n(\Theta_t)$$

where $\eta > 0$ is the **learning rate**. The gradient is:

$$\nabla_\Theta \hat{\mathcal{R}}_n(\Theta) = \frac{1}{n} \sum_{i=1}^n \nabla_\Theta\, \ell(y_i,\ f_\Theta(\mathbf{x}_i))$$

Computing this requires a forward pass and backward pass through the entire dataset. For modern datasets ($n \sim 10^9$ tokens), this is prohibitively expensive.

Stochastic Gradient Descent (SGD)

Instead of the full gradient, we approximate it using a random mini-batch $\mathcal{B}_t \subset \{1, \ldots, n\}$ of size B:

$$\mathbf{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_\Theta\, \ell(y_i,\ f_\Theta(\mathbf{x}_i))$$

This is an unbiased estimator: $\mathbb{E}_{\mathcal{B}_t}[\mathbf{g}_t] = \nabla_\Theta \hat{\mathcal{R}}_n(\Theta_t)$. The SGD update is:

$$\Theta_{t+1} = \Theta_t - \eta\, \mathbf{g}_t$$

The variance of $\mathbf{g}_t$ decreases as $O(1/B)$. Larger batches give lower-variance gradient estimates but cost more computation per step. In practice, B is chosen to saturate GPU parallelism (typically $B \in [32, 4096]$).

Remark 1. *For convex losses, SGD with a decaying learning rate $\eta_t \to 0$ (satisfying $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$) converges to the global minimum. For non-convex losses, SGD converges to a stationary point $\nabla \hat{\mathcal{R}}_n(\Theta) = \mathbf{0}$, which may be a local minimum or saddle point.*

Momentum

SGD oscillates in directions of high curvature. Momentum smooths the trajectory by accumulating a velocity:

$$\mathbf{v}_{t+1} = \mu\,\mathbf{v}_t + \mathbf{g}_t \tag{1}$$

$$\Theta_{t+1} = \Theta_t - \eta\,\mathbf{v}_{t+1} \tag{2}$$

where $\mu \in [0, 1)$ is the momentum coefficient (typically $\mu = 0.9$). The velocity $\mathbf{v}_t$ is an exponential moving average of past gradients. Unrolling the recursion:

$$\mathbf{v}_t = \sum_{s=0}^{t-1} \mu^{t-1-s}\,\mathbf{g}_s$$

Gradients that consistently point in the same direction accumulate; gradients that oscillate cancel. This accelerates convergence along low-curvature directions.

Proposition 2 (Effective learning rate)**.** *In a direction where the gradient is constant g at every step, the steady-state velocity is $v = g/(1-\mu)$. The effective step size is $\eta/(1-\mu)$, an amplification by factor $1/(1-\mu)$. For $\mu = 0.9$, this is $10\times$.*

Proof. At steady state, $v = \mu v + g$, so $v(1-\mu) = g$ and $v = g/(1-\mu)$. □

Adaptive Learning Rates

AdaGrad

Different parameters may have vastly different gradient magnitudes. AdaGrad (Duchi et al., 2011) adapts the learning rate per-parameter by dividing by the root sum of squared past gradients:

$$\Theta_{t+1,j} = \Theta_{t,j} - \frac{\eta}{\sqrt{G_{t,j} + \epsilon}}\,g_{t,j}$$

where $G_{t,j} = \sum_{s=0}^{t} g_{s,j}^2$ is the accumulated squared gradient for parameter j.

Problem. $G_{t,j}$ grows monotonically, so the effective learning rate $\eta/\sqrt{G_{t,j}}$ decays to zero. For non-convex problems, training may stop prematurely.

RMSProp

RMSProp (Hinton, 2012) fixes AdaGrad's decay by using an exponential moving average instead of a sum:

$$r_{t+1,j} = \beta\,r_{t,j} + (1-\beta)\,g_{t,j}^2$$

$$\Theta_{t+1,j} = \Theta_{t,j} - \frac{\eta}{\sqrt{r_{t+1,j} + \epsilon}}\,g_{t,j}$$

with $\beta \in (0, 1)$ (typically $\beta = 0.999$). The denominator now tracks the recent scale of gradients, not their entire history.

Adam

Adam (Kingma & Ba, 2015) combines momentum with adaptive learning rates. It maintains two exponential moving averages:

First moment (mean of gradient):

$$\mathbf{m}_{t+1} = \beta_1 \, \mathbf{m}_t + (1 - \beta_1) \, \mathbf{g}_t$$

Second moment (mean of squared gradient):

$$\mathbf{v}_{t+1} = \beta_2 \, \mathbf{v}_t + (1 - \beta_2) \, \mathbf{g}_t^2$$

where $\mathbf{g}_t^2$ denotes element-wise squaring.

Bias correction. Since $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$, the estimates are biased toward zero in the early steps. To see this, unroll the first moment:

$$\mathbf{m}_t = (1 - \beta_1) \sum_{s=1}^{t} \beta_1^{t-s} \, \mathbf{g}_s \tag{3}$$

Taking expectations (assuming $\mathbb{E}[\mathbf{g}_s] = \mathbf{g}$ is constant):

$$\mathbb{E}[\mathbf{m}_t] = \mathbf{g} \, (1 - \beta_1) \sum_{s=1}^{t} \beta_1^{t-s} = \mathbf{g} \, (1 - \beta_1^t)$$

So $\mathbb{E}[\mathbf{m}_t]$ underestimates $\mathbf{g}$ by a factor of $(1 - \beta_1^t)$. The bias-corrected estimates are:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

The Adam update is:

$$\boxed{\Theta_{t+1} = \Theta_t - \eta \, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}}$$

where division is element-wise. Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Remark 3. *The denominator $\sqrt{\hat{\mathbf{v}}_t}$ approximates $\mathbb{E}[|\mathbf{g}|]$ (the RMS of the gradient). So Adam's effective update is approximately $\eta \cdot \mathit{sign}(\mathbf{g})$ in magnitude, with the direction given by the momentum. This means Adam's step size is roughly independent of the gradient magnitude, which helps when gradients vary by orders of magnitude across layers.*

Backpropagation

To compute $\nabla_\Theta \, \ell(y, f_\Theta(\mathbf{x}))$ we use the **chain rule** applied to the computation graph. Consider a network of L layers:

$$\mathbf{h}^{(0)} = \mathbf{x}, \qquad \mathbf{h}^{(\ell)} = \sigma\Big(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\Big), \qquad \ell = 1, \ldots, L$$

where σ is a nonlinear activation applied element-wise. The output is $\hat{y} = f_\Theta(\mathbf{x}) = \mathbf{h}^{(L)}$.

Forward Pass

Compute and store all intermediate activations $\mathbf{h}^{(0)}, \mathbf{h}^{(1)}, \ldots, \mathbf{h}^{(L)}$ and pre-activations $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$. This costs $O(\sum_\ell d_\ell \, d_{\ell-1})$ operations and requires storing all activations in memory.

Backward Pass

Define $\boldsymbol{\delta}^{(\ell)} = \frac{\partial \ell}{\partial \mathbf{z}^{(\ell)}} \in \mathbb{R}^{d_\ell}$ as the error signal at layer ℓ. The chain rule gives:

Output layer:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \ell}{\partial \mathbf{z}^{(L)}}$$

Hidden layers (propagating backward from $\ell + 1$ to ℓ):

$$\boldsymbol{\delta}^{(\ell)} = \Big(\mathbf{W}^{(\ell+1)\top}\boldsymbol{\delta}^{(\ell+1)}\Big) \odot \sigma'(\mathbf{z}^{(\ell)})$$

where $\odot$ denotes element-wise multiplication and σ' is the derivative of the activation.

Parameter gradients:

$$\frac{\partial \ell}{\partial \mathbf{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \, \mathbf{h}^{(\ell-1)\top} \tag{4}$$

$$\frac{\partial \ell}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \tag{5}$$

The backward pass has the same computational cost as the forward pass. Total cost of one gradient computation: two passes through the network.

Computational Graph Perspective

More generally, backpropagation is reverse-mode automatic differentiation on a directed acyclic graph (DAG). Each node v computes a function $v = f_v(\text{parents}(v))$. For a scalar loss ℓ at the root:

Forward: Evaluate all nodes in topological order.

Backward: For each node v in reverse topological order, compute:

$$\frac{\partial \ell}{\partial v} = \sum_{u \in \text{children}(v)} \frac{\partial \ell}{\partial u} \cdot \frac{\partial u}{\partial v}$$

The cost of the backward pass is at most a constant factor ($\approx 2\times$) of the forward pass. This is the key efficiency result: computing all $|\Theta|$ partial derivatives costs no more than a few forward evaluations, regardless of the number of parameters.

The Vanishing and Exploding Gradient Problem

Consider the gradient of the loss with respect to parameters in layer ℓ of an L-layer network. By the chain rule:

$$\frac{\partial \ell}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial \ell}{\partial \mathbf{z}^{(L)}} \cdot \prod_{k=\ell+1}^{L} \left(\frac{\partial \mathbf{z}^{(k)}}{\partial \mathbf{z}^{(k-1)}} \right) \cdot \frac{\partial \mathbf{z}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}}$$

The product of Jacobians $\prod_{k=\ell+1}^{L} \frac{\partial \mathbf{z}^{(k)}}{\partial \mathbf{z}^{(k-1)}}$ involves $L - \ell$ matrix multiplications. Each Jacobian has the form:

$$\frac{\partial \mathbf{z}^{(k)}}{\partial \mathbf{z}^{(k-1)}} = \mathbf{W}^{(k)} \operatorname{diag}\Big(\sigma'(\mathbf{z}^{(k-1)})\Big)$$

If the spectral norms $\|\mathbf{W}^{(k)} \operatorname{diag}(\sigma'(\mathbf{z}^{(k-1)}))\|$ are consistently less than 1, the product shrinks exponentially with depth: gradients **vanish**. If they are consistently greater than 1, the product grows exponentially: gradients **explode**.

Consequence. In deep networks, early layers receive gradients that are either negligibly small (learning stalls) or astronomically large (parameters diverge). This was the primary obstacle to training deep networks. Several remedies exist.

Remedies

- **ReLU activations**: $\sigma(z) = \max(0, z)$ has $\sigma'(z) = \mathbf{1}_{z>0}$, which is either 0 or 1. This avoids the multiplicative shrinkage caused by saturating activations like sigmoid ($\sigma'(z) \leq 1/4$).
- **Residual connections**: As discussed in the transformer chapter, $\mathbf{x}' = f(\mathbf{x}) + \mathbf{x}$ gives a Jacobian $\mathbf{I} + \frac{\partial f}{\partial \mathbf{x}}$, which is at least the identity. This provides a gradient highway through the network.
- **Careful initialization**: Choosing initial weights to preserve activation variance across layers (see §6).
- **Normalization layers**: LayerNorm, BatchNorm stabilize the distribution of activations (see §7).
- **Gradient clipping**: Truncating large gradients to prevent explosions (see §8).

Weight Initialization

The Problem

Consider a single layer $\mathbf{z} = \mathbf{W}\mathbf{h}$ with $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. If entries of $\mathbf{h}$ have variance $\operatorname{Var}(h_j) = \sigma_h^2$ and entries of $\mathbf{W}$ are i.i.d. with $\mathbb{E}[W_{ij}] = 0$ and $\operatorname{Var}(W_{ij}) = \sigma_W^2$, then:

$$z_i = \sum_{j=1}^{d_{\text{in}}} W_{ij}\, h_j$$

Assuming independence between W_{ij} and h_j:

$$\operatorname{Var}(z_i) = d_{\text{in}}\, \sigma_W^2\, \sigma_h^2$$

If $\sigma_W^2 > 1/d_{\text{in}}$, the variance grows layer by layer; if $\sigma_W^2 < 1/d_{\text{in}}$, it shrinks. Either way, after L layers, the variance is $(d_{\text{in}}\, \sigma_W^2)^L\, \sigma_x^2$, which diverges or collapses exponentially.

Xavier Initialization

Proposition 4 (Glorot & Bengio, 2010)**.** *To preserve variance in both the forward and backward pass, initialize:*

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{d_{in} + d_{out}}\right)$$

Proof. For the forward pass, $\mathrm{Var}(z_i) = d_{\text{in}}\,\sigma_W^2\,\sigma_h^2$. Setting $\mathrm{Var}(z_i) = \sigma_h^2$ requires $\sigma_W^2 = 1/d_{\text{in}}$. For the backward pass, an analogous argument gives $\sigma_W^2 = 1/d_{\text{out}}$. The Xavier initialization uses the harmonic mean: $\sigma_W^2 = 2/(d_{\text{in}} + d_{\text{out}})$. □

He Initialization

For ReLU activations, $\sigma(z) = \max(0, z)$ zeroes out roughly half the units. This halves the variance:

$$\mathrm{Var}(\sigma(z)) \approx \frac{1}{2}\mathrm{Var}(z)$$

To compensate, He et al. (2015) initialize:

$$W_{ij} \sim \mathcal{N}\left(0,\ \frac{2}{d_{\text{in}}}\right)$$

This ensures $\mathrm{Var}(\mathbf{h}^{(\ell)}) \approx \mathrm{Var}(\mathbf{h}^{(\ell-1)})$ through ReLU layers.

Normalization

Batch Normalization

Given a mini-batch $\{z_i\}_{i=1}^B$ of pre-activations at a single neuron, batch normalization (Ioffe & Szegedy, 2015) computes:

$$\mu_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B} z_i, \qquad \sigma_{\mathcal{B}}^2 = \frac{1}{B}\sum_{i=1}^{B}(z_i - \mu_{\mathcal{B}})^2 \tag{6}$$

$$\hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \tag{7}$$

$$\tilde{z}_i = \gamma\,\hat{z}_i + \beta \tag{8}$$

where γ, β are learned parameters. The normalization is over the *batch dimension*: different examples at the same neuron.

Effect. At every layer, activations are re-centered and re-scaled, preventing the internal covariate shift that can slow training. BatchNorm also introduces stochasticity (through $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$), acting as a mild regularizer.

Problem for sequences. BatchNorm normalizes across examples, which requires a batch. At inference, running statistics are used instead. For variable-length sequences, positions near the end of long sequences may have very few examples in the batch, making the statistics unreliable.

Layer Normalization

Layer normalization (Ba et al., 2016) normalizes over the *feature dimension* of a single example, as described in the transformer chapter:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma + \epsilon} + \beta, \qquad \mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2}$$

LayerNorm depends only on a single token's vector, not on the batch. This makes it the standard choice for transformer-based models.

RMSNorm

RMSNorm (Zhang & Sennrich, 2019) simplifies LayerNorm by removing the mean centering:

$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}, \qquad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2}$$

This saves the mean computation and is used in many modern LLMs (e.g., LLaMA, Gemma).

Gradient Clipping

Even with good initialization and normalization, gradient explosions can occur, especially in early training or with large learning rates. Gradient clipping truncates the gradient when its norm exceeds a threshold.

Norm clipping. Given a threshold $c > 0$:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \le c \\ c\,\dfrac{\mathbf{g}}{\|\mathbf{g}\|} & \text{if } \|\mathbf{g}\| > c \end{cases}$$

This preserves the direction of the gradient but caps its magnitude. The clipped gradient satisfies $\|\mathbf{g}\| \le c$ always.

Remark 5. *The norm $\|\mathbf{g}\|$ is computed over all parameters jointly (the global norm), not per-layer. This ensures all parameters are scaled by the same factor, preserving relative gradient magnitudes across layers.*

Regularization

Minimizing the training loss alone leads to overfitting: the model memorizes the training set but performs poorly on unseen data. Regularization introduces an inductive bias favoring simpler models.

L_2 Regularization (Weight Decay)

We add a penalty proportional to the squared norm of the parameters:

$$\hat{\mathcal{R}}_n^{\text{reg}}(\Theta) = \hat{\mathcal{R}}_n(\Theta) + \frac{\lambda}{2}\|\Theta\|_2^2$$

The gradient becomes $\nabla\hat{\mathcal{R}}_n(\Theta) + \lambda\Theta$, so the update is:

$$\Theta_{t+1} = \Theta_t - \eta\left(\nabla\hat{\mathcal{R}}_n(\Theta_t) + \lambda\Theta_t\right) = (1-\eta\lambda)\Theta_t - \eta\,\nabla\hat{\mathcal{R}}_n(\Theta_t)$$

At each step, the parameters are shrunk by a factor $(1-\eta\lambda)$ before the gradient step. This is why L_2 regularization is called **weight decay**.

Remark 6. *For Adam, L_2 regularization and weight decay are **not** equivalent. L_2 regularization adds $\lambda\Theta$ to the gradient, which then gets normalized by the adaptive learning rate. True weight decay (AdamW, Loshchilov & Hutter, 2019) decouples the decay:*

$$\Theta_{t+1} = (1-\eta\lambda)\Theta_t - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon}$$

Dropout

During training, each hidden unit is independently zeroed out with probability p:

$$\tilde{h}_j = \begin{cases} 0 & \text{with probability } p \\ h_j/(1-p) & \text{with probability } 1-p \end{cases}$$

The scaling by $1/(1-p)$ ensures $\mathbb{E}[\tilde{h}_j] = h_j$, so the expected output is unchanged.

Interpretation. At each training step, dropout samples a random sub-network. Training with dropout is approximately equivalent to training an exponentially large ensemble of sub-networks, with shared weights. At test time, the full network (with no dropout) approximates the ensemble average.

In transformers. Dropout is applied after the attention weights and after each sub-layer (before the residual addition). Modern large-scale LLMs often use very low or zero dropout, relying on the massive dataset as an implicit regularizer.

Learning Rate Schedules

The learning rate η is the most important hyperparameter. A constant learning rate is suboptimal: too large early on causes divergence; too small later slows convergence.

Step Decay

Multiply η by a constant factor at fixed epochs:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

where $\gamma < 1$ is the decay factor and s is the step size in epochs.

Cosine Annealing

A smooth schedule that decays the learning rate from $\eta_{\max}$ to $\eta_{\min}$ over T steps:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T}\right)\right)$$

This is the standard schedule for training LLMs. Combined with a linear warmup (see §10.3), it yields the **warmup-cosine** schedule.

Linear Warmup

Start with a very small learning rate and increase linearly over T_w warmup steps:

$$\eta_t = \eta_{\max} \cdot \frac{t}{T_w}, \qquad t \leq T_w$$

Warmup prevents large gradient updates in the first few steps, when the model is far from any reasonable region and Adam's running statistics have not yet stabilized.

The Warmup-Cosine Schedule

Combining warmup and cosine decay:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \dfrac{t}{T_w} & t \leq T_w \\ \eta_{\min} + \dfrac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\dfrac{\pi(t - T_w)}{T - T_w}\right)\right) & t > T_w \end{cases}$$

This is the dominant schedule in modern LLM training.

Overfitting and Generalization

Bias-Variance Decomposition

For squared loss, the expected test error of an estimator $\hat{f}$ trained on D_n decomposes as:

$$\mathbb{E}\Big[(y - \hat{f}(\mathbf{x}))^2\Big] = \underbrace{\mathrm{Var}(y|\mathbf{x})}_{\text{irreducible error}} + \underbrace{\Big(\mathbb{E}[\hat{f}(\mathbf{x})] - f^*(\mathbf{x})\Big)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}(\hat{f}(\mathbf{x}))}_{\text{variance}} \tag{9}$$

where $f^*(\mathbf{x}) = \mathbb{E}[y|\mathbf{x}]$ is the Bayes-optimal predictor. The bias measures approximation error (how well the model class can fit the truth); the variance measures estimation error (how much $\hat{f}$ fluctuates across training sets).

In neural networks: Very large networks have low bias (they can approximate any function) but potentially high variance. Regularization, early stopping, and more training data reduce variance.

Early Stopping

Monitor the loss on a held-out validation set D_{val}. Stop training when the validation loss begins to increase:

$$t^* = \arg\min_t \hat{\mathcal{R}}_{\text{val}}(\Theta_t)$$

Early stopping is an implicit form of regularization: it limits the effective complexity of the model by restricting how far Θ moves from the initialization.

Double Descent

Classical learning theory predicts that test error follows a U-shaped curve as model complexity increases: decreasing (underfitting region), then increasing (overfitting region). Modern deep learning exhibits a more complex phenomenon:

In the **interpolation regime** (where the model has enough parameters to fit the training data exactly), the test error first spikes at the interpolation threshold, then *decreases* again as the model grows even larger. This is called **double descent**. The second descent is attributed to the implicit bias of gradient-based optimization, which selects smooth interpolators among the many that fit the data.

Practical Training Pipeline

A modern training run proceeds as follows:

1. *Initialize* parameters using He or Xavier initialization.
2. *Choose optimizer*: AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$.
3. *Choose schedule*: linear warmup over T_w steps, then cosine decay.
4. *For each step t*:
   (a) Sample mini-batch $\mathcal{B}_t$ of size B.
   (b) Forward pass: compute $\hat{\mathcal{R}}_{\mathcal{B}_t}(\Theta_t)$.
   (c) Backward pass: compute $\nabla_\Theta \hat{\mathcal{R}}_{\mathcal{B}_t}(\Theta_t)$.
   (d) Clip gradient: $\|\mathbf{g}\| \leq c$.
   (e) AdamW update with current learning rate η_t and weight decay λ.
5. *Evaluate* on held-out set periodically.

Training LLMs and Transformers

The Pretraining Objective

We now specialize the general training framework to the decoder-only transformer language model from the previous chapter. The model $f_\Theta : \mathcal{V}^* \to [0,1]^V$ maps a sequence of tokens to a distribution over the vocabulary. The training objective is maximum likelihood, or equivalently, minimization of the cross-entropy loss.

Let $\mathbf{w} = (w_1, w_2, \ldots, w_T)$ be a training sequence. The loss is:

$$\mathcal{L}(\Theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\Theta(w_t \mid w_{1:t-1})$$

where $p_\Theta(w_t \mid w_{1:t-1}) = \mathrm{softmax}\left(\mathbf{h}_t^{(L)}\,\mathbf{E}^\top\right)_{w_t}$ is the model's predicted probability for the true next token.

Connection to perplexity. The perplexity of the model on a sequence is:

$$\mathrm{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log p_\Theta(w_t \mid w_{1:t-1})\right)$$

A perplexity of k means the model is, on average, as uncertain as a uniform distribution over k tokens. Lower perplexity is better. If the model assigns probability 1 to every correct next token, $\mathrm{PPL} = 1$.

Remark 7. *The cross-entropy loss is an upper bound on the true entropy of the language:*

$$H(\textit{language}) \leq \mathcal{L}(\Theta)$$

The gap $\mathcal{L}(\Theta) - H(\textit{language})$ is the KL divergence between the true distribution and the model, plus the approximation error from the finite context window.

Data: Tokenization and Batching

Tokenization

The raw text must be converted to a sequence of integers from $\mathcal{V} = \{1, \ldots, V\}$. Modern LLMs use **subword tokenization**: neither character-level (too long, too little semantic content per token) nor word-level (vocabulary too large, cannot handle unseen words).

Byte Pair Encoding (BPE). Start with a vocabulary of individual bytes (or characters). Repeatedly merge the most frequent adjacent pair into a new token. After M merges, the vocabulary has $256 + M$ tokens (for byte-level BPE).

The merge procedure:

1. Initialize vocabulary $\mathcal{V}_0 = \{\text{all bytes}\}$.
2. For $m = 1, \ldots, M$:
   (a) Count all adjacent pairs (a, b) in the tokenized corpus.

(b) Let (a^*, b^*) be the most frequent pair.

(c) Create new token $c = a^* b^*$ and add to vocabulary: $\mathcal{V}_m = \mathcal{V}_{m-1} \cup \{c\}$.

(d) Replace all occurrences of (a^*, b^*) with c in the corpus.

3. Final vocabulary $\mathcal{V} = \mathcal{V}_M$ with $V = 256 + M$.

Common vocabulary sizes: GPT-2 uses $V = 50{,}257$; LLaMA uses $V = 32{,}000$; GPT-4 uses $V \approx 100{,}000$.

Packing Sequences into Batches

Training data consists of a massive corpus tokenized into a single stream of tokens $w_1, w_2, \ldots, w_N$ (with N on the order of trillions). This stream is sliced into non-overlapping chunks of length T (the **context length**):

$$\mathbf{w}^{(1)} = (w_1, \ldots, w_T), \qquad \mathbf{w}^{(2)} = (w_{T+1}, \ldots, w_{2T}), \qquad \ldots$$

A mini-batch consists of B such chunks, processed in parallel. The input to the model is a tensor of shape $[B \times T]$.

For each chunk, the model predicts every position in parallel (thanks to the causal mask), yielding $B \cdot T$ next-token predictions per step. The total number of training tokens seen is:

$$N_{\text{tokens}} = B \cdot T \cdot (\text{number of steps})$$

Gradient Computation for the Cross-Entropy Loss

Let us derive the gradient of the cross-entropy loss with respect to the logits, as this is the starting point for backpropagation through the transformer.

At position t, the model produces logits $\mathbf{u} = (u_1, \ldots, u_V) \in \mathbb{R}^V$ and the loss is:

$$\mathcal{L}_t = -\log \frac{\exp(u_{w_t})}{\sum_{k=1}^{V} \exp(u_k)} = -u_{w_t} + \log \sum_{k=1}^{V} \exp(u_k)$$

The gradient with respect to u_j is:

$$\frac{\partial \mathcal{L}_t}{\partial u_j} = -\mathbf{1}_{j=w_t} + \frac{\exp(u_j)}{\sum_{k=1}^{V} \exp(u_k)} \tag{10}$$

$$= p_j - \mathbf{1}_{j=w_t} \tag{11}$$

where $p_j = \text{softmax}(\mathbf{u})_j$. In vector form:

$$\boxed{\frac{\partial \mathcal{L}_t}{\partial \mathbf{u}} = \mathbf{p} - \mathbf{e}_{w_t}}$$

where $\mathbf{e}_{w_t}$ is the one-hot vector for the true token. The gradient at the output layer is the difference between the predicted distribution and the true distribution. From here, standard backpropagation propagates the gradient through the transformer layers.

Mixed-Precision Training

Modern GPUs have specialized hardware (tensor cores) that perform matrix multiplications in reduced precision far faster than in full precision.

FP32 (32-bit float): 1 sign bit, 8 exponent bits, 23 mantissa bits. Full precision.

FP16 (16-bit float): 1 sign bit, 5 exponent bits, 10 mantissa bits. Half the memory, $\sim 2\times$ faster matmuls, but limited range ($\sim 6 \times 10^4$ max).

BF16 (Brain Float 16): 1 sign bit, 8 exponent bits, 7 mantissa bits. Same range as FP32, but lower precision. The standard choice for LLM training.

The Mixed-Precision Recipe

Store and update parameters in FP32 (**master weights**). For each training step:

1. Cast weights to BF16.
2. Forward pass in BF16.
3. Compute loss in FP32 (for numerical stability of log-sum-exp).
4. Backward pass in BF16.
5. Update master weights (FP32) using FP32 gradients (accumulated from BF16).

The FP32 master copy is necessary because weight updates $\Delta\Theta$ can be very small relative to Θ. In half precision, the update may round to zero ($\Theta + \Delta\Theta = \Theta$ in FP16 when $|\Delta\Theta| < \epsilon_{\text{mach}}|\Theta|$). The FP32 copy preserves these small updates.

Loss scaling. For FP16 (not BF16), gradients can underflow to zero. Multiply the loss by a large constant S before the backward pass (scaling all gradients by S), then divide the gradients by S before the optimizer step. S is adjusted dynamically: increased when no overflow occurs, decreased when it does.

Distributed Training

Training a large LLM requires distributing computation across many GPUs. The three axes of parallelism are:

Data Parallelism

Each GPU holds a full copy of the model. The batch is split across GPUs: GPU g processes mini-batch $\mathcal{B}^{(g)}$. After the backward pass, gradients are averaged across GPUs using an **all-reduce** operation:

$$\mathbf{g} = \frac{1}{G}\sum_{g=1}^{G} \mathbf{g}^{(g)}$$

All GPUs then perform the same parameter update, keeping models synchronized. This is mathematically equivalent to training on a single GPU with batch size $G \cdot B$.

Limitation. The entire model must fit in each GPU's memory. For models with $> 10^{10}$ parameters, this is infeasible on current hardware.

Tensor Parallelism

Split individual weight matrices across GPUs. For example, in the FFN layer $\mathbf{Y} = \mathrm{ReLU}(\mathbf{X}\mathbf{W}_1)\mathbf{W}_2$:

Split $\mathbf{W}_1 \in \mathbb{R}^{d\times 4d}$ column-wise across G GPUs: GPU g holds $\mathbf{W}_1^{(g)} \in \mathbb{R}^{d\times 4d/G}$. Each GPU computes $\mathrm{ReLU}(\mathbf{X}\mathbf{W}_1^{(g)}) \in \mathbb{R}^{N\times 4d/G}$.

Split $\mathbf{W}_2 \in \mathbb{R}^{4d\times d}$ row-wise: GPU g holds $\mathbf{W}_2^{(g)} \in \mathbb{R}^{4d/G\times d}$. Each GPU computes a partial output, then all-reduce sums them:

$$\mathbf{Y} = \sum_{g=1}^{G} \mathrm{ReLU}(\mathbf{X}\mathbf{W}_1^{(g)})\, \mathbf{W}_2^{(g)}$$

This requires one all-reduce per layer, introducing communication overhead proportional to model size and inversely proportional to network bandwidth.

Pipeline Parallelism

Assign different transformer blocks to different GPUs: GPU g holds blocks $\ell_{g-1}+1, \ldots, \ell_g$. The forward pass proceeds sequentially through GPUs; the backward pass returns in reverse.

Problem: bubble overhead. While GPU g is computing the forward pass, GPUs $g+1, \ldots, G$ are idle. Micro-batching alleviates this: split the batch into M micro-batches and pipeline them, so multiple GPUs are active simultaneously.

ZeRO and FSDP

Zero Redundancy Optimizer (ZeRO) partitions optimizer states, gradients, and parameters across GPUs, rather than replicating them. **ZeRO Stage 3** (equivalently, Fully Sharded Data Parallelism / FSDP) shards all three:

- Each GPU stores only $1/G$ of the parameters, gradients, and optimizer states.
- Before each layer's forward/backward pass, the full parameters are gathered via all-gather.
- After the backward pass, gradients are scattered via reduce-scatter.

Memory per GPU scales as $O(|\Theta|/G)$ instead of $O(|\Theta|)$, enabling training of models that would not fit on a single GPU.

Scaling Laws

Definition 8 (Neural Scaling Laws)**.** *The test loss $\mathcal{L}$ of a transformer language model follows power laws in model size N (non-embedding parameters), dataset size D (tokens), and compute budget C (FLOP):*

$$\mathcal{L}(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \mathcal{L}_\infty \tag{12}$$

$$\mathcal{L}(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} + \mathcal{L}_\infty \tag{13}$$

$$\mathcal{L}(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} + \mathcal{L}_\infty \tag{14}$$

where N_c, D_c, C_c are scale constants, $\alpha_N, \alpha_D, \alpha_C$ are exponents, and $\mathcal{L}_\infty$ is the irreducible entropy of the language.

Kaplan et al. (2020) estimated $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, and found that performance is primarily a function of total compute, largely independent of model shape (depth vs. width) within reasonable ranges.

Compute-Optimal Scaling (Chinchilla)

Given a fixed compute budget C (in FLOP), how should we allocate between model size N and training tokens D? The total compute is approximately:

$$C \approx 6ND$$

(the factor 6 accounts for forward + backward, each requiring ~ 2 FLOP per parameter per token, times a constant factor).

Kaplan et al. (2020) suggested scaling N faster than D: for a $10\times$ increase in compute, increase N by $5.5\times$ and D by $1.8\times$. This led to models like GPT-3, which was trained on $\sim$ 300B tokens but had 175B parameters.

Hoffmann et al. (2022, "Chinchilla") revised this, finding that N and D should scale *equally*:

$$N_{\text{opt}} \propto C^{0.5}, \qquad D_{\text{opt}} \propto C^{0.5}$$

The optimal ratio is approximately $D \approx 20N$: for every parameter, train on about 20 tokens. Chinchilla (70B parameters, 1.4T tokens) matched the performance of Gopher (280B parameters, 300B tokens) at $4\times$ lower inference cost.

Remark 9. *Chinchilla-optimal scaling minimizes loss for a fixed training compute budget. In practice, inference cost matters too. A smaller model serving millions of requests may be more expensive than training a larger model. Recent practice (LLaMA, etc.) thus "over-trains" on more tokens than Chinchilla-optimal, accepting slightly higher training cost for a smaller, cheaper-to-serve model.*

Training Stability

Training large transformers is delicate. Common failure modes:

Loss Spikes

The training loss occasionally spikes upward, sometimes by orders of magnitude, before recovering. Causes include:

- Rare, anomalous data batches.
- Numerical instability (overflow in attention logits, underflow in normalization).
- Interaction between the learning rate and accumulated optimizer state.

Mitigation. Many training runs skip batches that produce anomalously large gradients, or restart from a checkpoint just before the spike with a reduced learning rate.

The Role of Gradient Clipping

For transformer LLMs, gradient clipping (typically $c = 1.0$) is not optional but *necessary*. Without it, occasional large gradients destroy the optimizer's running statistics, leading to cascading instabilities. The combination of gradient clipping + AdamW + warmup is what makes stable training of hundred-billion-parameter models feasible.

Architectural Stability Choices

Several architectural decisions improve stability:

- **Pre-norm** (norm before sub-layer) instead of post-norm reduces gradient magnitude at initialization.
- **RMSNorm** instead of LayerNorm is slightly more stable and efficient.
- **QK-norm**: Applying LayerNorm to queries and keys before computing attention scores prevents the logits from growing with depth.
- **Independent head scaling**: Initializing the output projection $\mathbf{W}^O$ with smaller scale (e.g., $1/\sqrt{2L}$) ensures the residual stream is not dominated by any single layer.

Post-Training: From Language Model to Assistant

Pretraining produces a model that predicts the next token. This is not yet an assistant: it will continue any text, including harmful, incorrect, or unhelpful continuations. Post-training aligns the model to follow instructions and be helpful.

Supervised Fine-Tuning (SFT)

Given a dataset of (prompt, desired response) pairs $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{M}$, fine-tune the pretrained model on the same cross-entropy loss, but only over the response tokens:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{M}\sum_{i=1}^{M}\frac{1}{|\mathbf{y}^{(i)}|}\sum_{t=1}^{|\mathbf{y}^{(i)}|}\log p_\Theta\Big(y_t^{(i)} \mid \mathbf{x}^{(i)}, y_{1:t-1}^{(i)}\Big)$$

This teaches the model the format of helpful responses. The learning rate is typically much smaller than in pretraining ($\sim 10\times$ to $100\times$ lower), and training runs for only a few epochs to avoid catastrophic forgetting of pretrained knowledge.

Reinforcement Learning from Human Feedback (RLHF)

SFT teaches format but not *quality*. RLHF uses human preferences to further improve the model.

Step 1: Reward model. Collect human comparisons: given a prompt $\mathbf{x}$ and two responses $\mathbf{y}_w \succ \mathbf{y}_l$ (where $\mathbf{y}_w$ is preferred), train a reward model $r_\phi(\mathbf{x}, \mathbf{y}) \in \mathbb{R}$ using the Bradley-Terry model:

$$p(\mathbf{y}_w \succ \mathbf{y}_l) = \sigma(r_\phi(\mathbf{x}, \mathbf{y}_w) - r_\phi(\mathbf{x}, \mathbf{y}_l))$$

where σ is the sigmoid. The loss is:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}[\log \sigma(r_\phi(\mathbf{x}, \mathbf{y}_w) - r_\phi(\mathbf{x}, \mathbf{y}_l))]$$

Step 2: Policy optimization. The SFT model is the initial policy π_Θ. Optimize the expected reward while staying close to the SFT model π_{ref}:

$$\max_\Theta \ \mathbb{E}_{\mathbf{x}\sim\mathcal{D},\, \mathbf{y}\sim\pi_\Theta(\cdot|\mathbf{x})}[r_\phi(\mathbf{x}, \mathbf{y})] - \beta\, \text{KL}(\pi_\Theta \,\|\, \pi_{\text{ref}})$$

The KL penalty prevents the policy from deviating too far from the pretrained model, which would lead to reward hacking (generating outputs that score high on the reward model but are actually low quality).

This is typically optimized with Proximal Policy Optimization (PPO). The KL-constrained optimum has a closed form:

$$\pi^*(\mathbf{y}|\mathbf{x}) \propto \pi_{\text{ref}}(\mathbf{y}|\mathbf{x})\, \exp\left(\frac{1}{\beta}\, r_\phi(\mathbf{x}, \mathbf{y})\right)$$

Direct Preference Optimization (DPO)

DPO (Rafailov et al., 2023) eliminates the need for an explicit reward model. The key insight: the closed-form optimum above implies that the reward is:

$$r(\mathbf{x}, \mathbf{y}) = \beta \log \frac{\pi_\Theta(\mathbf{y}|\mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}|\mathbf{x})} + \beta \log Z(\mathbf{x})$$

Substituting into the Bradley-Terry loss, the partition function $Z(\mathbf{x})$ cancels, yielding:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\Theta(\mathbf{y}_w|\mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}_w|\mathbf{x})} - \beta \log \frac{\pi_\Theta(\mathbf{y}_l|\mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}_l|\mathbf{x})}\right)\right]$$

DPO directly optimizes the policy using preference data, without training a separate reward model or running RL. It is simpler, more stable, and widely adopted.

Compute Requirements

FLOP Estimation

For a transformer with N non-embedding parameters, a single forward pass on one token requires approximately $2N$ FLOP (one multiply-add per parameter). The backward pass requires approximately $4N$ FLOP ($2\times$ the forward). Total per token per training step:

$$C_{\text{token}} \approx 6N \text{ FLOP}$$

For a training run on D tokens:

$$C_{\text{total}} \approx 6ND \text{ FLOP}$$

Example. LLaMA-2 70B trained on 2T tokens: $C \approx 6 \times 70 \times 10^9 \times 2 \times 10^{12} = 8.4 \times 10^{23}$ FLOP. At A100 peak throughput ($\sim 3 \times 10^{14}$ FLOP/s BF16) with $\sim 40\%$ hardware utilization, this requires:

$$\frac{8.4 \times 10^{23}}{3 \times 10^{14} \times 0.4} \approx 7 \times 10^9 \text{ seconds} \approx 220 \text{ GPU-years}$$

Or equivalently, $\sim$1000 A100 GPUs for $\sim$80 days.

Memory Requirements

For a model with N parameters in mixed precision:

- Model parameters (BF16): $2N$ bytes
- Gradients (BF16): $2N$ bytes
- Optimizer states (FP32 master weights + first/second moments for Adam): $4N + 4N + 4N = 12N$ bytes
- Total: $\sim 16N$ bytes per model copy

For a 70B parameter model: $16 \times 70 \times 10^9 \approx 1.1$ TB. This exceeds any single GPU's memory, necessitating the distributed strategies of §5.

Activation memory (for storing intermediate values during the forward pass) is proportional to $B \cdot T \cdot d \cdot L$ and can dominate for long sequences. **Activation checkpointing** reduces this by recomputing activations during the backward pass instead of storing them, trading $\sim 33\%$ more compute for a significant reduction in memory.

Summary of Key Design Choices

Component	Standard Choice	Why
Optimizer	AdamW	Adaptive LR + decoupled decay
Schedule	Warmup + cosine decay	Stable early training, smooth decay
Precision	BF16 + FP32 master	Speed + range + update fidelity
Normalization	RMSNorm (pre-norm)	Stable gradients, efficient
Initialization	$\sim\mathcal{N}(0, 1/\sqrt{d})$ per layer	Variance preservation
Gradient clipping	Global norm ≤ 1.0	Prevents explosion
Parallelism	FSDP + tensor parallel	Memory efficiency + speed
Tokenizer	BPE ($V \sim$ 32–128K)	Subword compression
Post-training	SFT $\rightarrow$ DPO/RLHF	Format $\rightarrow$ quality alignment